07 - 데이터베이스 및 스토리지 시스템

개요

Geth의 데이터베이스 및 스토리지 시스템은 블록체인 데이터의 영구 저장과 효율적인 검색을 담당하는 핵심 컴포넌트입니다. 이 시스템은 세 가지 주요 레이어로 구성됩니다:

1. **Key-Value 데이터베이스 레이어**: LevelDB 또는 Pebble 기반의 영구 저장소
2. **Merkle Patricia Trie 레이어**: 상태 데이터의 암호학적 검증 가능한 구조
3. **스냅샷 시스템**: 빠른 상태 접근을 위한 최적화 레이어

이 시스템은 블록체인의 모든 상태 데이터, 트랜잭션, 블록 정보를 안전하고 효율적으로 관리합니다.

핵심 구조체 및 인터페이스

1. 데이터베이스 인터페이스 계층

ethdb.Database 인터페이스

```
// Database는 key-value 저장소와 ancient 체인 저장소 모두에 접근하는
// 고수준 데이터베이스 인터페이스입니다
type Database interface {
KeyValueStore
AncientStore
}
```

```go
// KeyValueStore는 기본적인 key-value 저장소 기능을 제공합니다
type KeyValueStore interface {
KeyValueReader
KeyValueWriter
KeyValueStater
KeyValueSyncer
KeyValueRangeDeleter
Batcher
Iteratee
Compacter
io.Closer
}
```

핵심 메서드들

- `Get(key []byte) ([]byte, error)` : 키에 해당하는 값 조회
- `Put(key []byte, value []byte) error` : 키-값 쌍 저장
- `Delete(key []byte) error` : 키 삭제
- `Has(key []byte) (bool, error)` : 키 존재 여부 확인
- `NewBatch() Batch` : 배치 쓰기를 위한 배치 객체 생성

2. LevelDB 구현체

leveldb.Database 구조체

```go
type Database struct {
```

```
fn string // 파일명 (리포팅용)
db *leveldb.DB // LevelDB 인스턴스


// 메트릭 수집을 위한 미터들
compTimeMeter *metrics.Meter // 압축 시간 측정
compReadMeter *metrics.Meter // 압축 중 읽기 데이터 측정
compWriteMeter *metrics.Meter // 압축 중 쓰기 데이터 측정
diskSizeGauge *metrics.Gauge // 데이터베이스 크기 추적
diskReadMeter *metrics.Meter // 디스크 읽기 측정
diskWriteMeter *metrics.Meter // 디스크 쓰기 측정
quitChan chan chan error // 메트릭 수집 중단을 위한 채널
log log.Logger // 컨텍스트 로거
}
```

주요 메서드 구현

```
// New는 LevelDB 래퍼 객체를 생성합니다
func New(file string, cache int, handles int, namespace string, readonly b


// Get은 주어진 키가 key-value 저장소에 있으면 해당 값을 조회합니다
func (db *Database) Get(key []byte) ([]byte, error) {
dat, err := db.db.Get(key, nil)
if err != nil {
```

```
return  nil, err
}
return  dat, nil
}

// Put은 주어진 값을 key-value 저장소에 삽입합니다
func (db *Database) Put(key []byte, value []byte) error {
return  db.db.Put(key, value, nil)
}
```

3. Pebble 구현체

pebble.Database 구조체

```
type Database struct {
fn  string  // 파일명 (리포팅용)
db *pebble.DB // Pebble 저장 엔진
namespace  string  // 메트릭 네임스페이스

// 성능 메트릭들
compTimeMeter *metrics.Meter
diskSizeGauge *metrics.Gauge
liveMemTablesGauge *metrics.Gauge // 라이브 메모리 테이블 수
zombieMemTablesGauge *metrics.Gauge // 좀비 메모리 테이블 수
```

```
blockCacheHitGauge *metrics.Gauge // 블록 캐시 히트 수
blockCacheMissGauge *metrics.Gauge // 블록 캐시 미스 수
writeOptions *pebble.WriteOptions
}
```

Pebble 최적화 설정

```
// New는 Pebble DB 래퍼 객체를 생성합니다
func New(file string, cache int, handles int, namespace string, readonly b
opt := &pebble.Options{
Cache: pebble.NewCache(int64(cache * 1024 * 1024)),
MaxOpenFiles: handles,
MemTableSize: uint64(memTableSize),
// 더 빠른 압축을 위해 모든 CPU 사용
MaxConcurrentCompactions: runtime.NumCPU,
// 레벨별 옵션 설정
Levels: []pebble.LevelOptions{
{TargetFileSize: 2 * 1024 * 1024, FilterPolicy: bloom.FilterPolicy(10)},
{TargetFileSize: 4 * 1024 * 1024, FilterPolicy: bloom.FilterPolicy(10)},
// ... 더 많은 레벨들
},
}
```

```
}
```

Merkle Patricia Trie 시스템

1. 기본 Trie 구조

trie.Trie 구조체

```
// Trie는 Merkle Patricia Trie를 나타냅니다
type Trie struct {
root node // 루트 노드
owner common.Hash // 트라이 소유자 식별자

committed  bool  // 커밋 작업 완료 여부
unhashed  int  // 마지막 해싱 이후 삽입된 리프 수
uncommitted  int  // 마지막 커밋 이후 업데이트 수

reader *Reader // 노드 검색을 위한 핸들러
// 트라이 수정사항 추적을 위한 트레이서들
opTracer *opTracer
prevalueTracer *PrevalueTracer
}
```

핵심 메서드들

```go
// New는 제공된 트라이 ID와 읽기 전용 데이터베이스로 트라이 인스턴스를 생성합니다
func  New(id *ID, db database.NodeDatabase) (*Trie, error)


// Get은 키에 해당하는 값을 반환합니다
func (t *Trie) Get(key []byte) []byte


// Update는 키를 값과 연결합니다
func (t *Trie) Update(key, value []byte) error


// Delete는 트라이에서 키를 제거합니다
func (t *Trie) Delete(key []byte) error


// Commit은 트라이를 데이터베이스에 쓰고 새로운 루트 해시를 반환합니다
func (t *Trie) Commit(collectLeaf  bool) (common.Hash, *trienode.NodeSet, error)
```

2. 노드 타입들

노드 인터페이스와 구현체들

```go
type node interface {
cache() (hashNode, bool)
encode(w rlp.EncoderBuffer)
fstring(string) string
```

```go
}


// fullNode는 17개의 자식을 가진 브랜치 노드입니다
type fullNode struct {
Children [17]node // 실제 트라이 노드 데이터
flags nodeFlag
}


// shortNode는 키-값 또는 키-노드 쌍을 나타냅니다
type shortNode struct {
Key []byte
Val node
flags nodeFlag
}


// hashNode는 다른 노드의 해시 참조입니다
type hashNode []byte


// valueNode는 실제 값 데이터를 포함합니다
type valueNode []byte
```

3. Secure Trie (State Trie)

StateTrie 구조체

```go
// StateTrie는 키 해싱을 사용하는 트라이 래퍼입니다
type StateTrie struct {
trie Trie // 기본 트라이
db database.NodeDatabase // 노드 데이터베이스
preimages preimageStore // 프리이미지 저장소
secKeyCache  map[common.Hash][]byte  // 보안 키 캐시
}
```

주요 메서드들

```go
// NewStateTrie는 기존 루트 노드로부터 트라이를 생성합니다
func  NewStateTrie(id *ID, db database.NodeDatabase) (*StateTrie, error)

// Get은 키에 대한 값을 반환합니다 (키는 자동으로 해싱됩니다)
func (t *StateTrie) Get(key []byte) []byte {
return  t.trie.Get(t.hashKey(key))
}

// Update는 키를 값과 연결합니다 (키는 자동으로 해싱됩니다)
func (t *StateTrie) Update(key, value []byte) error {
hk := t.hashKey(key)
if  t.preimages != nil {
t.preimages.InsertPreimage(map[common.Hash][]byte{
```

```
common.BytesToHash(hk): key,
})
}
return  t.trie.Update(hk, value)
}


// hashKey는 키를 keccak256으로 해싱합니다
func (t *StateTrie) hashKey(key []byte) []byte {
hash := crypto.Keccak256(key)
t.secKeyCache[common.BytesToHash(hash)] = key
return  hash
}
```

스냅샷 시스템

1. 스냅샷 트리 구조

snapshot.Tree 구조체

```
// Tree는 상태 스냅샷의 전체 집합을 관리합니다
type Tree struct {
config Config // 스냅샷 설정
diskdb ethdb.KeyValueStore // 스냅샷 저장을 위한 영구 데이터베이스
triedb *triedb.Database // 트라이 접근을 위한 인메모리 캐시
```

```
layers  map[common.Hash]snapshot // 알려진 모든 레이어 컬렉션
lock sync.RWMutex

onFlatten  func() // 테스트 훅
}
```

주요 메서드들

```
// New는 영구 key-value 저장소에서 기존 스냅샷을 로드하려고 시도합니다
func  New(config Config, diskdb ethdb.KeyValueStore, triedb *triedb.Database, ro

// Snapshot은 주어진 블록 루트에 속하는 스냅샷을 조회합니다
func (t *Tree) Snapshot(blockRoot common.Hash) Snapshot {
t.lock.RLock()
defer  t.lock.RUnlock()
return  t.layers[blockRoot]
}

// Update는 새로운 블록의 상태 변경사항으로 스냅샷을 업데이트합니다
func (t *Tree) Update(blockRoot, parentRoot common.Hash, accounts  map[common.Ha
```

2. 디스크 레이어

diskLayer 구조체

```go
// diskLayer는 key-value 저장소 위에 구축된 저수준 영구 스냅샷입니다
type diskLayer struct {
diskdb ethdb.KeyValueStore // 기본 스냅샷을 포함하는 Key-value 저장소
triedb *triedb.Database // 재구성 목적의 트라이 노드 캐시
cache *fastcache.Cache // 직접 접근을 위한 캐시

root common.Hash // 기본 스냅샷의 루트 해시
stale  bool  // 레이어가 오래되었음을 나타내는 신호

genMarker []byte  // 초기 레이어 생성 중 인덱싱된 상태 마커
genPending  chan  struct{} // 생성 완료 알림 채널
genAbort  chan  chan *generatorStats // 생성 중단 알림 채널

lock sync.RWMutex
}
```

주요 메서드들

```go
// Account는 스냅샷에서 특정 해시와 연관된 계정을 직접 조회합니다
func (dl *diskLayer) Account(hash common.Hash) (*types.SlimAccount, error) {
data, err := dl.AccountRLP(hash)
if  err != nil {
return  nil, err
```

```
}
if len(data) == 0 {
return nil, nil
}
account := new(types.SlimAccount)
if err := rlp.DecodeBytes(data, account); err != nil {
panic(err)
}
return account, nil
}

// Storage는 특정 계정 내에서 특정 해시와 연관된 저장소 데이터를 직접 조회합니다
func (dl *diskLayer) Storage(accountHash, storageHash common.Hash) ([]byte, erro
```

3. Diff 레이어

diffLayer 구조체

```
// diffLayer는 상태 스냅샷 위에서 블록 실행 후 만들어진 수정사항 컬렉션을 나타냅니다
type diffLayer struct {
origin *diskLayer // 블룸 미스 시 직접 사용할 기본 디스크 레이어
parent snapshot // 이 레이어에 의해 수정된 부모 스냅샷

root common.Hash // 이 스냅샷 diff가 속하는 루트 해시
```

```
stale atomic.Bool // 레이어가 오래되었음을 나타내는 신호


accountData  map[common.Hash][]byte  // 직접 조회를 위한 키 계정들
storageData  map[common.Hash]map[common.Hash][]byte  // 직접 조회를 위한 키 저장소
accountList []common.Hash // 반복을 위한 계정 목록
storageList  map[common.Hash][]common.Hash // 반복 조회를 위한 저장소 슬롯 목록


diffed *bloomfilter.Filter // 디스크 레이어까지의 모든 diff 항목을 추적하는 블룸 필터


lock sync.RWMutex
}
```

주요 프로세스

1. 데이터베이스 초기화 과정

```
// 1. LevelDB 초기화 예시
func  initializeLevelDB() {
// 캐시와 파일 핸들 설정
cache := 512  // MB
handles := 256
// LevelDB 인스턴스 생성
db, err := leveldb.New("./chaindata", cache, handles, "eth/db/chaindata/", false
```

```go
if err != nil {
log.Fatal("Failed to open database", "err", err)
}
// 메트릭 수집 시작 (백그라운드에서 자동 실행)
// db.meter() 고루틴이 자동으로 시작됨
}


// 2. Pebble 초기화 예시
func initializePebble() {
// 더 나은 성능을 위한 Pebble 설정
db, err := pebble.New("./chaindata", 1024, 512, "eth/db/chaindata/", false)
if err != nil {
log.Fatal("Failed to open pebble database", "err", err)
}
// 비동기 쓰기 모드로 설정되어 더 나은 성능 제공
}
```

2. Trie 생성 및 업데이트 과정

```go
// 1. State Trie 생성
func createStateTrie() {
// 트라이 ID 설정
id := &trie.ID{
```

```
StateRoot: stateRoot,
Owner: common.Hash{}, // 메인 상태 트라이
Root: rootHash,
}
// State Trie 생성
stateTrie, err := trie.NewStateTrie(id, nodeDB)
if  err != nil {
log.Fatal("Failed to create state trie", "err", err)
}
// 계정 정보 업데이트
account := &types.StateAccount{
Nonce: 1,
Balance: big.NewInt(1000000),
Root: emptyRoot,
CodeHash: emptyCodeHash,
}
accountData, _ := rlp.EncodeToBytes(account)
stateTrie.Update(address.Bytes(), accountData)
// 트라이 커밋
newRoot, nodeSet, err := stateTrie.Commit(true)
if  err != nil {
log.Fatal("Failed to commit trie", "err", err)
}
```

```
}
```

3. 스냅샷 시스템 동작 과정

```
// 1. 스냅샷 트리 초기화
func initializeSnapshot() {
config := snapshot.Config{
CacheSize: 256,
Recovery: true,
NoBuild: false,
AsyncBuild: false,
}
// 스냅샷 트리 생성
snapshotTree, err := snapshot.New(config, diskDB, trieDB, rootHash)
if err != nil {
log.Fatal("Failed to create snapshot tree", "err", err)
}
// 특정 블록의 스냅샷 조회
snap := snapshotTree.Snapshot(blockRoot)
if snap != nil {
// 계정 정보 빠른 조회
account, err := snap.Account(accountHash)
if err != nil {
```

```go
log.Error("Failed to get account from snapshot", "err", err)
}
// 저장소 데이터 빠른 조회
storage, err := snap.Storage(accountHash, storageHash)
if err != nil {
log.Error("Failed to get storage from snapshot", "err", err)
}
}
}

// 2. 스냅샷 업데이트 과정
func updateSnapshot(tree *snapshot.Tree, blockRoot, parentRoot common.Hash) {
// 블록 실행으로 변경된 계정들
accounts := make(map[common.Hash][]byte)
accounts[accountHash] = accountData
// 블록 실행으로 변경된 저장소들
storage := make(map[common.Hash]map[common.Hash][]byte)
storage[accountHash] = make(map[common.Hash][]byte)
storage[accountHash][storageHash] = storageData
// 스냅샷 업데이트
err := tree.Update(blockRoot, parentRoot, accounts, storage)
if err != nil {
log.Error("Failed to update snapshot", "err", err)
```

```
}

}
```

최적화 및 성능

1. 데이터베이스 최적화

LevelDB 최적화

```
// 압축 최적화 설정

options := &opt.Options{

Filter: filter.NewBloomFilter(10), // 블룸 필터 사용

DisableSeeksCompaction: true, // Seek 압축 비활성화

BlockCacheCapacity: cache / 2 * opt.MiB, // 블록 캐시 설정

WriteBuffer: cache / 4 * opt.MiB, // 쓰기 버퍼 설정

}


// 배치 쓰기를 통한 성능 향상

batch := db.NewBatch()

for  key, value := range  data {

batch.Put(key, value)

}

batch.Write() // 한 번에 모든 변경사항 커밋
```

Pebble 최적화

```
// 메모리 테이블 최적화
memTableLimit := 4
memTableSize := cache * 1024 * 1024 / 2 / memTableLimit

opt := &pebble.Options{
MemTableSize: uint64(memTableSize),
MemTableStopWritesThreshold: memTableLimit,
MaxConcurrentCompactions: runtime.NumCPU, // 병렬 압축
L0CompactionThreshold: 2, // L0 압축 임계값 낮춤
}
```

2. Trie 최적화

노드 캐싱

```
// 트라이 노드 캐싱을 통한 성능 향상
type nodeCache struct {
cache  map[common.Hash]node
lock sync.RWMutex
}

func (nc *nodeCache) get(hash common.Hash) (node, bool) {
```

```go
nc.lock.RLock()
defer nc.lock.RUnlock()
n, ok := nc.cache[hash]
return n, ok
}

func (nc *nodeCache) set(hash common.Hash, n node) {
nc.lock.Lock()
defer nc.lock.Unlock()
nc.cache[hash] = n
}
```

배치 업데이트

```go
// 여러 키-값 쌍을 한 번에 업데이트
func batchUpdateTrie(trie *trie.StateTrie, updates map[string][]byte) error {
for key, value := range updates {
if err := trie.Update([]byte(key), value); err != nil {
return err
}
}
// 한 번에 커밋
_, _, err := trie.Commit(true)
return err
```

```
}
```

3. 스냅샷 최적화

블룸 필터 활용

```
// Diff 레이어에서 블룸 필터를 사용한 빠른 존재 여부 확인
func (dl *diffLayer) hasAccount(hash common.Hash) bool {
// 블룸 필터로 빠른 체크
if !dl.diffed.Contains(accountBloomHash(hash)) {
return false // 확실히 없음
}
// 블룸 필터 히트 시 실제 데이터 확인
_, exists := dl.accountData[hash]
return  exists
}
```

계층적 캐싱

```
// 디스크 레이어에서 fastcache를 사용한 메모리 캐싱
func (dl *diskLayer) getAccountWithCache(hash common.Hash) ([]byte, error) {
// 먼저 메모리 캐시 확인
if  blob, found := dl.cache.HasGet(nil, hash[:]);  found {
snapshotCleanAccountHitMeter.Mark(1)
```

```
return  blob, nil
}
// 캐시 미스 시 디스크에서 읽기
blob := rawdb.ReadAccountSnapshot(dl.diskdb, hash)
// 캐시에 저장
dl.cache.Set(hash[:], blob)
snapshotCleanAccountMissMeter.Mark(1)
return  blob, nil
}
```

디버깅 및 모니터링

1. 메트릭 수집

데이터베이스 메트릭

```
// LevelDB 메트릭 예시
type DatabaseMetrics struct {
CompactionTime *metrics.Meter // 압축 시간
CompactionRead *metrics.Meter // 압축 중 읽기
CompactionWrite *metrics.Meter // 압축 중 쓰기
DiskSize *metrics.Gauge // 디스크 사용량
DiskRead *metrics.Meter // 디스크 읽기
DiskWrite *metrics.Meter // 디스크 쓰기
```

```
WriteDelay *metrics.Meter // 쓰기 지연
}

// 메트릭 수집 루프
func (db *Database) meter(refresh time.Duration) {
for {
var  stats leveldb.DBStats
db.db.Stats(&stats)
// 메트릭 업데이트
db.diskSizeGauge.Update(stats.LevelSizes.Sum())
db.compTimeMeter.Mark(stats.LevelDurations.Sum().Nanoseconds())
time.Sleep(refresh)
}
}
```

스냅샷 메트릭

```
// 스냅샷 시스템 메트릭
var (
snapshotCleanAccountHitMeter = metrics.NewRegisteredMeter("state/snapshot/clean/
snapshotCleanAccountMissMeter = metrics.NewRegisteredMeter("state/snapshot/clean
snapshotDirtyAccountHitMeter = metrics.NewRegisteredMeter("state/snapshot/dirty/
snapshotFlushAccountItemMeter = metrics.NewRegisteredMeter("state/snapshot/flush
```

```
)
```

2. 로깅 및 디버깅

데이터베이스 상태 로깅

```
// 데이터베이스 통계 출력
func (db *Database) Stat() (string, error) {
var  stats leveldb.DBStats
if  err := db.db.Stats(&stats);  err != nil {
return  "", err
}
message := fmt.Sprintf("Read(MB):%.5f Write(MB):%.5f\n",
float64(stats.IORead)/1048576.0,
float64(stats.IOWrite)/1048576.0)
message += fmt.Sprintf("BlockCache(MB):%.5f FileCache:%d\n",
float64(stats.BlockCacheSize)/1048576.0,
stats.OpenedTablesCount)
return  message, nil
}
```

트라이 디버깅

```
// 트라이 구조 시각화
```

```go
func (t *Trie) String() string {
if t.root == nil {
return "empty trie"
}
return t.root.fstring("")
}

// 노드별 디버그 정보
func (n *fullNode) fstring(ind string) string {
resp := fmt.Sprintf("[\n%s ", ind)
for i, node := range &n.Children {
if node == nil {
resp += fmt.Sprintf("%s: <nil> ", indices[i])
} else {
resp += fmt.Sprintf("%s: %v", indices[i], node.fstring(ind+" "))
}
}
return resp + fmt.Sprintf("\n%s] ", ind)
}
```

관련 컴포넌트

1. 블록체인 코어와의 연동

- `core.BlockChain` 이 상태 변경 시 트라이 업데이트
- `state.StateDB` 가 트라이를 통해 계정 상태 관리
- 블록 커밋 시 스냅샷 시스템 업데이트

2. P2P 네트워킹과의 연동

- 상태 동기화 시 트라이 노드 요청/응답
- 스냅샷 동기화를 통한 빠른 초기 동기화
- 트라이 증명을 통한 상태 검증

3. RPC API와의 연동

- `eth_getBalance`, `eth_getStorageAt` 등의 API에서 트라이 조회
- 스냅샷을 통한 빠른 상태 조회 응답
- 디버그 API에서 데이터베이스 통계 제공

이 데이터베이스 및 스토리지 시스템은 Geth의 모든 데이터 관리의 기반이 되며, 성능과 안정성이 전체 시스템에 직접적인 영향을 미치는 핵심 컴포넌트입니다.